Operating Systems – Midterm Examination
April 16, 2013

Total: 101 points

1. Hardware devices signal interrupts when important events occur (e.g., hard-disk data are available):
   (a) (5 points) Show the advantage of interrupt-based operations over busy-waiting.
   (b) (10 points) Describe the process of hardware interrupt handling. You need to mention device controller, interrupt-request line, state saving, interrupt vector, interrupt-specific handler.

2. (5 points) What role do device controllers and device drivers play in a computer system?

3. (10 points) What is the dual-mode operation and how does it ensure the proper execution of the operating system?

4. System calls:
   (a) (10 points) How does an API function invoke a system call? You need to mention software interrupt, dual-mode, system call number (index), and table of code pointer.
   (b) (5 points) Explain why programs mostly access system services via API functions rather than making system call directly.
   (c) (5 points) Describe an appropriate method used to pass "**MANY**" parameters to the operating system during system calls.

5. (5 points) What are the advantages of the modular kernel approach over the layered and microkernel design techniques?

6. Show the state of a process:
   (a) (3 points) after it is selected by a long-term scheduler
   (b) (3 points) before it is selected by a short-term scheduler
   (c) (3 points) after it is selected by a short-term scheduler
   (d) (3 points) before it invokes a blocking system call

(e) (3 points) after it invokes a blocking system call

(f) (3 points) after the blocking system call is complete

(g) (3 points) after it is interrupted by a hardware interrupt

7. Multithread model:

(a) (5 points) Describe the pros and cons of the many-to-one thread model.

(b) (5 points) Describe the pros and cons of the one-to-one thread model.

8 (15 points) Complete the following C++ program which serves as a simple shell interface. The shell accepts user commands and then executes each command in a separate process. The shell allows the child process to run in the background (or concurrently) if the command ends with '&'. For simplicity, we assume that users never enter invalid commands and no arguments follow a command.

getchar

```
int main(void)
{
   char  inputBuffer[80];  // buffer to hold command
entered
   bool background; // true if a command is followed by
'&'

   while (  )
   {
      cout << endl << " COMMAND->" ;
      pid_t pid;

      // step 1
      // accept user command and check if the command is
      // followed by '&'


      // step 2
      // fork a child process, if background == false,
      // the parent will wait
      // the child process invoke exec(inputBuffer) to
      // execute user command


   } // end of while

} // end of main
```

!EOF

if == '&'
background = true.

& → childprocess 執行

沒有& → 一起